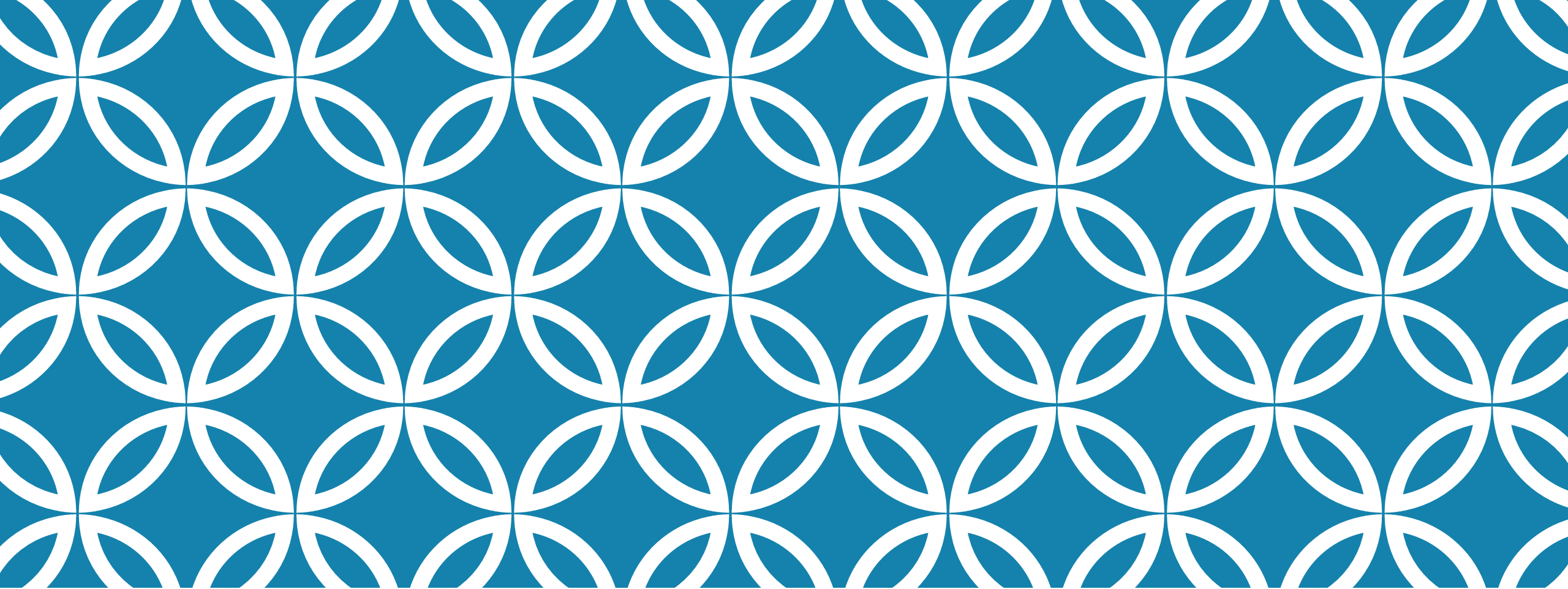




OBJECT ORIENTED PROGRAMMING

WEEK 3

05-10 SEP, 2022

Instructor:

Abdul Aziz
Assistant Professor
(School of Computing)
National University- FAST (KHI Campus)

ACKNOWLEDGMENT

- Publish material by Virtual University of Pakistan.
- Publish material by Deitel & Deitel.
- Publish material by Robert Lafore.



WEEK 3, DAY 1

Constructor and its types.

WHAT IS A CONSTRUCTOR?

“It is a member function which initializes a class”

A constructor has:

- (i) *A member function*
- (ii) *It has the same name as the class name*
- (iii) *Called automatically when an object is created*
- (iv) *Never returns a value (no return type)*
- (v) *May have arguments*
- (vi) *Only called once in object lifeline*

EXAMPLE

```
class rectangle {  
    private:  
        float height;  
        float width;  
        int xpos, ypos;  
    public:  
        rectangle();           // constructor  
        void draw();           // draw member function  
        void pos(int, int);     // position member function  
        void move(int, int);    // move member function  
};
```

COMMENTS ON CONSTRUCTORS

A constructor is called automatically whenever a new instance of a class is created.

It ensures that a memory is allocated to the object.

If you do not specify a constructor, the compiler generates a default constructor for you (expects no parameters and has an empty body).

Used for object initialization.

COMMENTS ON CONSTRUCTORS (CONT.)

```
class rectangle {  
    private:  
        float height;  
        float width;  
        int xpos, ypos;  
    public:  
        rectangle(){  
            height=0, width=0, xpos=0,ypos=0;  
        }  
        void pos(int i, int j){  
            xpos=i;  
            ypos=j;  
        }  
};
```

TYPES OF CONSTRUCTOR

1- Default Constructor

2- User Define Constructor

- NULL Constructor
- Parameterized Constructor
- Copy Constructor

TYPES OF CONSTRUCTORS

You can have more than one constructor in a class, as long as each has a different list of arguments.

public:

```
rectangle(){
    height=0, width=0, xpos=0,ypos=0;
}
rectangle (float i, float j){
    height=i;
    width=j;
}

rectangle (float i){
    height=i;
}
```

```
void main()
{
    rectangle rc1(3.0, 2.0);
    rectangle rc2;
    rectangle rc3(10);
}
```



WEEK 3, DAY 2

Copy Constructor

WHAT IS A COPY CONSTRUCTOR?

It is a member function which initializes an object using another object of the same class.

A copy constructor has the following general function prototype:

```
class_name (const class_name&);
```

```
class rectangle {  
    private:  
        float height;  
        float width;  
        int xpos;  
        int ypos;  
    public:  
        rectangle(float, float);          // constructor  
        rectangle(const rectangle&); // copy constructor  
  
};
```

```
rectangle::rectangle(const rectangle& old_rc)
{
    height = old_rc.height;
    width = old_rc.width;
    xpos = old_rc.xpos;
    ypos = old_rc.ypos;
}
```

```
void main()
{
    rectangle rc1(3.0, 2.0);           // use constructor
    rectangle rc2(rc1);                // use copy constructor
    rectangle rc3 = rc1;               // alternative syntax for
                                      // copy constructor

    C++ statements;
}
```

DEFINING COPY CONSTRUCTORS IS VERY IMPORTANT

In the absence of a copy constructor, the C++ compiler builds a default copy constructor for each class which is doing a memberwise copy between objects.

Default copy constructors work fine unless the class contains pointer data members ... why???

```
#include <iostream.h>
#include <string.h>

class string {
private:
    char *s;
    int size;
public:
    string(char *); // constructor
    ~string();      // destructor
    void print();
    void copy(char *);
};

void string::print()
{
    cout << s << endl;
}
```

```
void string::copy(char *c)
{
    strcpy(s, c);
}
```

```
void main()
{
    string str1("George");
    string str2 = str1;    // default copy constructor
```

```
    str1.print();    // what is printed ?
    str2.print();
```

```
    str2.copy("Mary");
```

```
    str1.print();    // what is printed now ?
    str2.print();
}
```


DEFINING A COPY CONSTRUCTOR FOR THE ABOVE EXAMPLE:

```
class string {  
    private:  
        char *s;  
        int size;  
    public:  
        string(char *);           // constructor  
        ~string();               // destructor  
        string(const string&);    // copy constructor  
        void print();  
        void copy(char *);  
};
```

```
string::string(const string& old_str)
```

```
{
```

```
    size = old_str.size;
```

```
    s = new char[size+1];
```

```
    strcpy(s,old_str.s);
```

```
}
```

```
void main()
```

```
{
```

```
    string str1("George");
```

```
    string str2 = str1;
```

```
    str1.print(); // what is printed ?
```

```
    str2.print();
```

```
    str2.copy("Mary");
```

```
    str1.print(); // what is printed now ?
```

```
    str2.print();
```

```
}
```

Note: same results can be obtained by overloading the assignment operator.



WEEK 3, DAY 3

Destructor

WHAT IS A DESTRUCTOR?

It is a member function which deletes an object.

A destructor function is called automatically when the object goes out of scope:

- (1) the function ends
- (2) the program ends
- (3) a block containing temporary variables ends
- (4) a *delete* operator is called

A destructor has:

- (i) the same name as the class but is preceded by a tilde (~)
- (ii) no arguments and return no values

```
class string {
private:
    char *s;
    int size;
public:
    string(char *); // constructor
    ~string();      // destructor
};
```

```
string::string(char *c)
{
    size = strlen(c);
    s = new char[size+1];
    strcpy(s,c);
}
```

```
string::~~string()
{
    delete []s;
}
```

COMMENTS ON DESTRUCTORS

If you do not specify a destructor, the compiler generates a default destructor for you.

When a class contains a pointer to memory you allocate, **it is your responsibility** to release the memory before the class instance is destroyed.